

O ciclo de vida de uma requisição HTTP no Express

Rita de Cássia Rodrigues da Silva
Rogers Rocha Carvalho

Universidade Federal do Piauí – Campus Senador Helvídio Nunes de Barros

Bacharelado em Sistemas de Informação

Programação para a Web II – Prof. Evandro J. R. Silva

Framework: Express 5 (Node.js)

Tema: Tema 1 – O ciclo de vida de uma requisição HTTP no servidor

Resumo

Este relatório descreve o caminho percorrido por uma requisição HTTP em uma aplicação Express 5, desde a chegada dos bytes ao servidor até o envio da resposta ao cliente. São apresentados os conceitos de *middleware*, *pipeline*, *short-circuit*, *controller* e camada de serviço/modelo, e o fluxo é ilustrado por uma mini API com três middlewares (log, autenticação fictícia e tratamento de erro), um controller e um service. Os logs de execução mostram que, quando um middleware interrompe o pipeline antes do controller, nem o controller nem o service são executados, e que a formatação das respostas de erro pode ser centralizada em um único middleware de erro global.

1 Introdução e problema

O Express é o framework web mais utilizado do ecossistema Node.js. Ele é minimalista e não impõe uma arquitetura: oferece um roteador e um mecanismo de composição de funções (*middlewares*). Essa flexibilidade cria um problema didático e prático. Como o caminho da requisição é definido pela ordem em que o código registra as funções, muitos desenvolvedores usam o framework sem um modelo mental preciso do que acontece entre a chegada dos bytes no *socket* e o envio da resposta.

As consequências aparecem em bugs comuns:

- rotas que nunca são alcançadas porque um middleware anterior encerrou a requisição;
- `req.body` indefinido porque o *parser* foi registrado depois da rota;
- erros assíncronos que derrubam o processo ou deixam a requisição pendurada;
- expectativas erradas sobre o que o código depois de `next()` faz.

Objetivo. Descrever, com precisão, o caminho de uma requisição desde a chegada ao servidor até a resposta ao cliente, mapeando a ordem do pipeline, o papel dos middlewares, o momento de entrada do controller e da camada de serviço/modelo e o retorno da resposta, incluindo a comparação com o modelo cebola (*onion*). O trabalho responde, ainda, à pergunta central: *o que acontece se um middleware interromper o pipeline antes do controller, e quem é responsável por formatar a resposta de erro?*

Base. O relatório considera o Express 5, hoje a versão padrão publicada no npm, e aponta as diferenças em relação ao Express 4.x quando elas afetam o pipeline. A análise é documental [1, 2, 3, 4] e é ilustrada por uma mini API (Seção 4), cujos logs de execução servem como evidência do caminho percorrido pela requisição.

2 Conceitos

Middleware. É uma função com assinatura (`req, res, next`). Ela tem acesso ao objeto de requisição, ao de resposta e à função que passa o controle adiante. Pode executar código, alterar `req` e `res`, encerrar o ciclo (enviando resposta) ou chamar `next()`. Os middlewares de erro têm quatro parâmetros, (`err, req, res, next`), e o Express os distingue pela aridade (`fn.length === 4`).

Pipeline. É a sequência ordenada de camadas (*layers*) mantida na pilha interna do roteador. Cada `app.use()`, `app.get()`, `router.post()` etc. adiciona uma camada. A ordem de registro é a ordem de execução. Não há priorização automática.

Short-circuit (curto-circuito). Ocorre quando um middleware encerra a requisição sem chamar `next()` – respondendo diretamente ou desviando o fluxo com `next(err)` –, por exemplo num middleware de autenticação que rejeita a requisição com 401. Tudo o que estava registrado depois dele no fluxo normal deixa de executar para aquela requisição. Também existem curtos-circuitos parciais: `next('route')` pula os handlers restantes da rota atual, e `next('router')` sai do roteador atual.

Controller. É o handler final de uma rota. Traduz HTTP em chamadas de aplicação: extrai parâmetros (`req.params`, `req.query`, `req.body`), chama o serviço e converte o resultado em resposta (status, cabeçalhos, corpo). Não deve conter regras de negócio.

Service/Model layer. O *service* concentra as regras de negócio e orquestra operações. O *model* (ou repositório) encapsula o acesso a dados: ORM, queries, APIs externas. Ambas são independentes do HTTP, ou seja, não conhecem `req` nem `res`. O Express não as fornece; são convenção arquitetural do desenvolvedor [10].

3 Análise e discussão

3.1 Antes do Express: do socket ao `app.handle`

O Express não recebe bytes diretamente. O caminho anterior à sua atuação é [6]:

1. O cliente resolve o DNS, abre a conexão TCP (e o *handshake* TLS, se HTTPS) e envia a requisição.
2. O servidor `http.Server` do Node (criado por `http.createServer(app)` ou por `app.listen()`) lê o *socket*, faz o *parsing* da linha de requisição e dos cabeçalhos (via `llhttp`) e emite o evento `'request'` com `IncomingMessage (req)` e `ServerResponse (res)`.
3. O `app` do Express é, na verdade, uma função (`req, res`) registrada como *listener* desse evento. Ela chama `app.handle(req, res)`.

Nesse ponto o corpo da requisição ainda não foi lido. Ele chega como *stream* e só é consumido por middlewares específicos.

3.2 Preparação interna e ordem do pipeline

Antes das camadas do desenvolvedor, o Express prepara `req` e `res`. A forma como isso acontece mudou entre as versões:

- **Express 4.x:** `app.lazyrouter()` registra duas camadas internas no início da pilha. A camada `query` faz o *parsing* da query string e popula `req.query`; a camada `expressInit` define o cabeçalho `X-Powered-By`, liga `req` e `res` aos protótipos do Express (é isso que dá `res.json`, `res.status`, `req.get` etc.) e prepara `res.locals`.
- **Express 5:** `lazyrouter()` e essas duas camadas foram removidos. O trabalho do `expressInit` passou a ser feito diretamente dentro de `app.handle`, antes de o roteador ser chamado, e `req.query` tornou-se um *getter*, calculado sob demanda na primeira leitura (com o *parser* "simple" como padrão) [4]. O efeito é visível na mini API: a resposta traz o cabeçalho `X-Powered-By: Express` mesmo quando a requisição é interrompida no middleware de autenticação (Teste 1, Seção 4.1), porque ele é definido antes de qualquer camada do desenvolvedor.

Só depois entram as camadas do desenvolvedor, na ordem de registro. Uma ordem típica e recomendada é mostrada na Tabela 1.

Tabela 1: Ordem típica do pipeline de middlewares.

Pos.	Camada	Função
1	Segurança/infra (helmet, cors)	Cabeçalhos de segurança, política CORS (responde <i>preflight</i> OPTIONS e pode curto-circuitar)
2	Log (morgan)	Registra a requisição; captura dados no fim via eventos
3	compression	Envolve <code>res.write/end</code> para comprimir a saída
4	<code>express.json()</code> , <code>express.urlencoded()</code>	Lê o <i>stream</i> e preenche <code>req.body</code>
5	<code>cookie-parser</code> , <code>session</code>	Interpreta cookies e carrega a sessão
6	Rate limit	Pode responder 429 (curto-circuito)
7	Autenticação/autorização	Valida token; responde 401/403 ou anexa <code>req.user</code>
8	Arquivos estáticos (<code>express.static</code>)	Responde direto se o arquivo existe
9	<i>Routers</i> (<code>app.use('/api', router)</code>)	Casamento de rota, validação, controller
10	Handler de 404	Captura o que nenhuma rota tratou
11	Middleware de erro (4 argumentos)	Centraliza tratamento de exceções

A regra que explica a maioria dos bugs é que o registro define a ordem. Por exemplo, `express.json()` depois da rota resulta em `req.body === undefined`.

3.3 O que cada middleware pode fazer

Cada camada tem cinco possibilidades:

- **Inspecionar/modificar** `req` e `res` (anexar `req.user`, definir cabeçalhos, alterar `req.url`).
- **Executar efeitos colaterais** (log, métricas, abrir contexto de rastreamento).
- **Passar adiante** com `next()`.
- **Encerrar a resposta** (`res.send`, `res.json`, `res.end`, `res.redirect`), o que caracteriza o curto-circuito.
- **Sinalizar erro** com `next(err)`, o que faz o Express pular todas as camadas normais e ir ao próximo middleware de erro.

Há ainda os *hooks* de roteamento: `router.param()` executa antes dos handlers de uma rota que contenha determinado parâmetro (útil para carregar um recurso por `:id`).

3.4 Casamento de rota e entrada do controller

Ao chegar nas camadas de roteamento, o Express compara método e caminho (via `path-to-regexp`; a versão 8, usada no Express 5, alterou a sintaxe de curingas). Se há *match*, extrai `req.params` e executa a cadeia de handlers daquela rota. Uma rota pode ter vários handlers em sequência:

```
router.post('/pedidos',
  autenticar, // middleware de rota
  validar(pedidoSchema), // middleware de rota (pode curto-circuitar com 400)
  pedidoController.criar // controller: último handler
);
```

O **controller entra por último**, depois que todos os middlewares anteriores (globais e específicos da rota) chamaram `next()`. Neste momento o corpo já está *parseado*, o usuário autenticado e anexado a `req.user`, e os parâmetros da rota extraídos em `req.params`. O controller, portanto, não precisa ler cabeçalhos de autenticação nem fazer o *parsing* do corpo: ele apenas consome o que as camadas anteriores prepararam. Se nenhuma rota casar com o método e o caminho, a requisição segue pela pilha até o handler de 404, registrado depois de todas as rotas.

3.5 Quando a camada de serviço/modelo é chamada

O Express não sabe que o service existe. Ele é chamado explicitamente pelo controller, que extrai os dados de `req` e os repassa como valores simples:

```
async function buscar(req, res) {
  const id = Number(req.params.id);
  const pedido = await pedidoService.buscarPorId(id, req.user.id);
  res.status(200).json(pedido);
}
```

O service recebe apenas identificadores, aplica as regras de negócio (o pedido existe? pertence a este usuário?) e, num projeto real, chamaria um model/repositório para acessar o banco. Como não conhece `req` nem `res`, pode ser testado e reutilizado fora do contexto HTTP. Quando uma regra é violada, o service não monta resposta: ele lança um erro (`throw`), cujo caminho é discutido na Seção 3.7.

3.6 Como a resposta volta: Express *versus* modelo cebola

No modelo cebola (*onion*), adotado por Koa [7] e ASP.NET Core [8], cada middleware executa `await next()` e retoma sua execução quando todas as camadas internas terminam. A resposta atravessa as camadas na ordem inversa da ida (Figura 1).

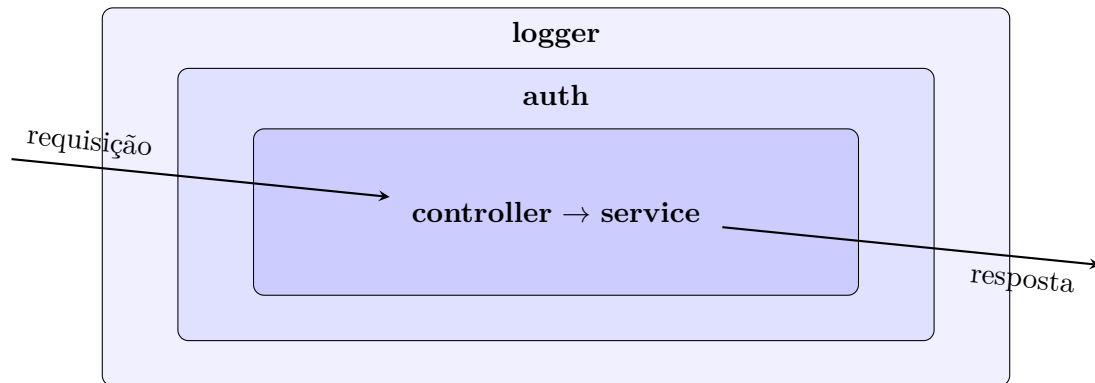


Figura 1: Modelo cebola: a requisição entra pelas camadas externas e a resposta sai por elas na ordem inversa (ida: `logger` → `auth` → `controller`; volta: `controller` → `auth` → `logger`).

O Express não é uma cebola literal. Sua função `next()` não retorna uma promessa que espere o restante do pipeline: ela apenas dispara a próxima camada. O código escrito depois de `next()` roda assim que a parte síncrona das camadas seguintes termina – se o `controller` for assíncrono (consulta ao banco, por exemplo), esse código roda antes de a resposta existir. É a “expectativa errada sobre o que o código depois de `next()` faz”, citada na introdução.

Além disso, a resposta é enviada no momento em que o `controller` chama `res.json()`; não há uma etapa em que as camadas anteriores “recebem” a resposta de volta para modificá-la. Para agir na volta, o Express usa eventos do objeto `res`. O middleware de log da mini API faz isso:

```
module.exports = function logger(req, res, next) {  
  const inicio = Date.now();  
  console.log(`[log] -> ${req.method} ${req.originalUrl}`);  
  res.on('finish', () => {  
    const ms = Date.now() - inicio;  
    console.log(`[log] <- ${req.method} ${req.originalUrl} ${res.statusCode} (  
      ${ms}ms)`);  
  });  
  next();  
};
```

O evento `'finish'` é disparado quando a resposta termina de ser enviada, seja ela de sucesso ou de erro. O resultado observável é o mesmo da cebola – a primeira camada a entrar é a última a registrar –, mas o mecanismo é baseado em eventos, e não no retorno de funções. Bibliotecas como `morgan` e `compression` usam a mesma estratégia.

3.7 Caminho dos erros

Quando uma camada chama `next(err)`, o Express pula todas as camadas normais restantes e procura o próximo middleware de erro (quatro parâmetros) [3]. No Express 5, se um handler assíncrono rejeitar a promessa (por exemplo, porque o service lançou um erro dentro de uma função `async`), o roteador chama `next(err)` automaticamente. No Express 4 isso não acontecia: a rejeição não era capturada e a requisição ficava pendurada, o que exigia `try/catch` em cada controller ou uma biblioteca auxiliar (`express-async-errors`).

O middleware de erro da mini API é o único ponto que decide o formato das respostas de erro:

```
module.exports = function errorHandler(err, req, res, next) {
  if (res.headersSent) return next(err);
  const status = err.status || 500;
  const code = err.code || 'ERRO_INTERNO';
  const message = status >= 500 ? 'Erro interno do servidor' : err.message;
  if (status >= 500) console.error(err);
  res.status(status).json({ erro: { codigo: code, mensagem: message } });
};
```

Três cuidados aparecem nele: (1) se os cabeçalhos já foram enviados, não é possível trocar a resposta, então o erro é repassado ao handler padrão do Express, que encerra a conexão; (2) erros sem status são tratados como 500; (3) em erros 5xx a mensagem real e o *stack trace* ficam só no log do servidor, sem vazarem para o cliente.

4 Implementação e resultados

Para ilustrar o ciclo descrito, foi implementada uma mini API em Express 5 que lista e busca pedidos de um usuário. A estrutura do projeto é:

src/	
app.js	montagem do pipeline
middlewares/logger.js	middleware 1: log
middlewares/auth.js	middleware 2: autenticação fictícia
middlewares/errorHandler.js	middleware 3: tratamento de erro
controllers/pedidoController.js	
services/pedidoService.js	
errors/HttpError.js	erro com status e código
demo.sh	script com as requisições de teste

O pipeline, em `app.js`, segue a ordem da Tabela 1:

```
app.use(logger); // log
app.use(express.json()); // parser do corpo
app.use(auth); // autenticação
app.get('/pedidos', pedidoController.listar); // rotas
app.get('/pedidos/:id', pedidoController.buscar);
app.use((req, res, next) => // 404
  next(new HttpError(404, 'ROTA_NAO_ENCONTRADA', 'Rota não existe')));
app.use(errorHandler); // erro (4 args)
```

O middleware de autenticação aceita apenas o cabeçalho `Authorization: Bearer token-valido`. Se o token for válido, anexa `req.user` e chama `next()`; caso contrário, chama `next(err)`. Ele não escreve a resposta: apenas sinaliza o erro.

```
const TOKENS = { 'token-valido': { id: 1, nome: 'Ana' } };

module.exports = function auth(req, res, next) {
  const header = req.get('Authorization') || '';
  const token = header.replace(/^Bearer\s+/i, '');
  const usuario = TOKENS[token];
  if (!usuario) {
    return next(new HttpError(401, 'NAO_AUTENTICADO', 'Token ausente ou inválido'));
  }
  req.user = usuario; // enriquece a requisição para as camadas seguintes
  next();
};
```

O service aplica as regras de negócio e lança erros quando elas são violadas:

```
async function buscarPorId(id, usuarioId) {
  const pedido = pedidos.find(p => p.id === id);
  if (!pedido) throw new HttpError(404, 'PEDIDO_NAO_ENCONTRADO', `Pedido ${id} não existe`);
  if (pedido.usuarioId !== usuarioId) throw new HttpError(403, 'ACESSO_NEGADO', 'Pedido pertence a outro usuário');
  return pedido;
}
```

4.1 Resultados

A API foi executada com `npm start`, e as requisições foram feitas com `curl`. A Tabela 2 resume os testes; em seguida são mostrados as respostas e os logs do servidor (os tempos em milissegundos variam a cada execução).

Tabela 2: Resumo dos testes realizados.

Teste	Requisição	Token	Status	Onde parou
1	GET /pedidos	ausente	401	middleware auth
2	GET /pedidos	válido	200	caminho completo
3	GET /pedidos/99	válido	404	service (pedido inexistente)
4	GET /pedidos/3	válido	403	service (pedido de outro usuário)

Teste 1 – sem token.

```
HTTP/1.1 401 Unauthorized
X-Powered-By: Express
Content-Type: application/json; charset=utf-8

{"erro":{"codigo":"NAO_AUTENTICADO","mensagem":"Token ausente ou inválido"}}

--- LOGS DO SERVIDOR ---
[log]    -> GET /pedidos
[auth]   verificando credenciais
```

```
[auth] FALHOU -> next(err), pipeline desviado para o errorHandler
[erro] formatando resposta 401 (NAO_AUTENTICADO)
[log] <- GET /pedidos 401 (6ms)
```

Teste 2 – token válido.

```
HTTP/1.1 200 OK
[{"id":1,"usuarioId":1,"item":"Teclado","total":150},
 {"id":2,"usuarioId":1,"item":"Mouse","total":80}]

--- LOGS DO SERVIDOR ---
[log] -> GET /pedidos
[auth] verificando credenciais
[controller] listar
[service] listarDoUsuario
[log] <- GET /pedidos 200 (1ms)
```

Teste 3 – pedido inexistente.

```
HTTP/1.1 404 Not Found
{"erro":{"codigo":"PEDIDO_NAO_ENCONTRADO","mensagem":"Pedido 99 não
  existe"}}

--- LOGS DO SERVIDOR ---
[log] -> GET /pedidos/99
[auth] verificando credenciais
[controller] buscar
[service] buscarPorId
[erro] formatando resposta 404 (PEDIDO_NAO_ENCONTRADO)
[log] <- GET /pedidos/99 404 (2ms)
```

Teste 4 – pedido de outro usuário.

```
HTTP/1.1 403 Forbidden
{"erro":{"codigo":"ACESSO_NEGADO","mensagem":"Pedido pertence a outro
  usuário"}}

--- LOGS DO SERVIDOR ---
[log] -> GET /pedidos/3
[auth] verificando credenciais
[controller] buscar
[service] buscarPorId
[erro] formatando resposta 403 (ACESSO_NEGADO)
[log] <- GET /pedidos/3 403 (1ms)
```

Análise dos resultados.

- No Teste 1 não aparecem as linhas `[controller]` nem `[service]`: o curto-circuito no `auth` impediu que essas camadas fossem executadas.
- Em todos os testes o log registra a entrada e a saída, inclusive nos erros, porque o logger foi a primeira camada registrada e escuta o evento `'finish'` – é o efeito “cebola” descrito na Seção 3.6.

- Os Testes 1, 3 e 4 têm origens diferentes (um middleware e o service), mas as respostas têm o mesmo formato JSON, porque todas passam pelo mesmo middleware de erro.
- Nos Testes 3 e 4 o erro é lançado dentro de uma função `async` do service, e o controller não tem `try/catch`. Mesmo assim o erro chega ao `errorHandler`, graças à captura automática de promessas rejeitadas do Express 5.

Análise dos resultados.

- No Teste 1 não aparecem as linhas `[controller]` nem `[service]`: o curto-circuito no `auth` impediu que essas camadas fossem executadas.
- Em todos os testes o log registra a entrada e a saída, inclusive nos erros, porque o logger foi a primeira camada registrada e escuta o evento `'finish'` – é o efeito “cebola” descrito na Seção 3.6.
- Os Testes 1, 3 e 4 têm origens diferentes (um middleware e o service), mas as respostas têm o mesmo formato JSON, porque todas passam pelo mesmo middleware de erro.
- Nos Testes 3 e 4 o erro é lançado dentro de uma função `async` do service, e o controller não tem `try/catch`. Mesmo assim o erro chega ao `errorHandler`, graças à captura automática de promessas rejeitadas do Express 5.

5 Pergunta central

O que acontece se um middleware interromper o pipeline antes do controller?

Todas as camadas normais registradas depois dele deixam de executar para aquela requisição – outros middlewares, o controller e, por consequência, o service. Isso é desejável: nenhuma regra de negócio ou acesso a dados ocorre para uma requisição não autenticada, e o servidor economiza processamento. As camadas que já executaram continuam ativas: o logger, que se inscreveu no evento `'finish'` antes da interrupção, ainda registra a saída com status 401 (Teste 1).

A interrupção pode acontecer de duas formas, com consequências diferentes:

- (a) o middleware responde diretamente, por exemplo `res.status(401).json(...)`. Nesse caso ele próprio formata a resposta, e nem o middleware de erro é executado;
- (b) o middleware chama `next(err)`. O Express pula as camadas normais e entrega o erro ao middleware de erro.

Quem é responsável por formatar a resposta de erro? Na forma (a), o próprio middleware que interrompeu – o que espalha a decisão sobre o formato por vários arquivos, e a consistência passa a depender da disciplina de cada desenvolvedor. Na forma (b), adotada na mini API, a responsabilidade é do middleware de erro global (`errorHandler`). O `auth` e o service apenas sinalizam o problema (`next(err)` ou `throw`) com um objeto `HttpError` que carrega status e código, mas nenhum deles escreve a resposta. Centralizar a formatação traz três vantagens:

- formato de erro consistente em toda a API (Testes 1, 3 e 4);

- um único ponto para impedir o vazamento de *stack trace* (erros **5xx** retornam apenas “Erro interno do servidor”);
- controllers e services mais simples, sem `try/catch`.

Se nenhum middleware de erro for registrado, o Express usa seu handler padrão, que responde em HTML e, fora do modo de produção (`NODE_ENV` diferente de `production`), inclui o *stack trace* na resposta – exatamente o vazamento que o `errorHandler` evita.

6 Conclusões e limitações

Conclusões. No Express, a ordem de registro define a ordem de execução, e não há priorização automática: grande parte dos bugs comuns citados na introdução decorre disso. O pipeline é linear na ida; o efeito de “volta” característico do modelo cebola não vem do retorno de `next()`, mas de eventos do objeto `res`, como `'finish'`. O controller só entra depois que todas as camadas anteriores chamaram `next()`, e o service só é chamado quando o controller o invoca explicitamente. O curto-circuito impede a execução do controller e do service, protegendo a aplicação e poupando recursos. Por fim, delegar os erros com `next(err)` a um middleware de erro global produz respostas consistentes e evita o vazamento de informações internas. No Express 5, a captura automática de promessas rejeitadas torna esse modelo mais simples e seguro do que no Express 4.

Limitações.

- A autenticação é fictícia (token fixo em memória); não foram usados JWT nem sessões reais.
- Não há banco de dados: os pedidos ficam em um vetor dentro do service, sem uma camada de model/repositório separada.
- O service lança `HttpError`, que carrega códigos HTTP. Numa arquitetura mais rigorosa ele lançaria erros de domínio puros (por exemplo, `PedidoNaoEncontrado`) e o middleware de erro faria o mapeamento para status HTTP [9].
- O caso de falha inesperada (erro 500) está tratado no código, mas não foi exercitado nos testes.
- O estudo cobre apenas o Express; a comparação com frameworks que implementam a cebola literalmente (Koa, ASP.NET Core) é conceitual.
- Não foram feitos testes automatizados nem medições de desempenho.

Referências

- [1] EXPRESS. *Using middleware*. Disponível em: <https://expressjs.com/en/guide/using-middleware.html>.
- [2] EXPRESS. *Writing middleware for use in Express apps*. Disponível em: <https://expressjs.com/en/guide/writing-middleware.html>.
- [3] EXPRESS. *Error handling*. Disponível em: <https://expressjs.com/en/guide/error-handling.html>.
- [4] EXPRESS. *Migrating to Express 5*. Disponível em: <https://expressjs.com/en/guide/migrating-5.html>.

- [5] EXPRESS. *Express 5.x API reference*. Disponível em: <https://expressjs.com/en/5x/api.html>.
- [6] NODE.JS. *HTTP*. Disponível em: <https://nodejs.org/api/http.html>.
- [7] KOA. *Koa – next generation web framework for Node.js*. Disponível em: <https://koajs.com/>.
- [8] MICROSOFT. *ASP.NET Core Middleware*. Disponível em: <https://learn.microsoft.com/aspnet/core/fundamentals/middleware/>.
- [9] FIELDING, R.; NOTTINGHAM, M.; RESCHKE, J. *RFC 9110: HTTP Semantics*. IETF, 2022. Disponível em: <https://www.rfc-editor.org/rfc/rfc9110>.
- [10] FOWLER, M. *Patterns of Enterprise Application Architecture*. Boston: Addison-Wesley, 2002.